

Mathematics Breakdown

Every Equation, Formula, and Constraint Used Across the Project

Dataset: Olist Brazilian E-Commerce Dataset (110,058 delivered order records; 592 days of daily revenue). This document lists every mathematical formula, equation, and constraint used across the three implemented frameworks (causal inference, kernel methods, forecasting), with the derivation, meaning, and verified numerical result for each.

Section 1 - Causal Inference Framework: Mathematics

1.1 Structural Data-Generating Process

$$\begin{aligned} Z &\sim \text{logN}(\mu_Z, \sigma_Z^2) \quad T = Z^a * \text{eps}_T, & \text{eps}_T &\sim \text{logN}(0, \sigma_T^2) \quad Y \\ &= Z^b * T^c * \text{eps}_Y, & \text{eps}_Y &\sim \text{logN}(0, \sigma_Y^2) \end{aligned}$$

Z is a positive-valued confounding variable (freight_ratio in this project). T is the treatment variable (approval_delay_hours). Y is the outcome (delivery_days). The parameter c represents the CAUSAL effect of T on Y; a and b represent the confounder's effects on T and Y respectively.

Fitted values from this project (log-space ordinary least squares, no intercept): a = -0.0695, b = -1.1399, c = 0.0692.

1.2 Joint Log-Normal Covariance

$$\begin{aligned} [\log Z, \log T, \log Y]^T &= A * [\log Z, \log \text{eps}_T, \log \text{eps}_Y]^T & A &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} a & 1 \\ b - a*c & -c \end{bmatrix} & \text{Sigma}_{[Z,T,Y]} &= A * \text{diag}(\sigma_Z^2, \sigma_T^2, \sigma_Y^2) * A^T \end{aligned}$$

Since an affine transformation of jointly Gaussian variables is itself jointly Gaussian, this shows Z, T, and Y are jointly log-normal, with a covariance matrix fully determined by the five parameters a, b, c, σ_Z^2 , σ_T^2 , σ_Y^2 .

1.3 Twin-Model Joint Distribution

$$\begin{aligned} p(z^*, t^*, y^*, \theta, D) &= [p_{\theta}(z^*) * q(t^*) * p_{\theta}(y^*|z^*, t^*)] \\ &\leftarrow \text{intervened world} & * [\text{PRODUCT}_i p_{\theta}(z_i) * p_{\theta}(t_i|z_i) * \\ &p_{\theta}(y_i|z_i, t_i)] &\leftarrow \text{observed world} & * p(\theta) \end{aligned}$$

This equation specifies ONE joint probability distribution spanning both the real (observed) world and a hypothetical (intervened) world, connected by shared parameters theta. $q(t^*)$ is the intervention distribution -- here set to a Dirac delta, $q(t^*) = \delta(t^* - t_{\text{target}})$, representing a deterministic assignment.

1.4 Closed-Form Interventional Expectation

$$E[Y^* | \text{do}(T^*=t^*)] = (t^*)^{(-c)} * \exp(b*\mu_Z + (b^2*\sigma_Z^2 + \sigma_Y^2) / 2)$$

This is the single most important formula in the causal-inference component: it converts the intervention question ("what happens to Y if we force T to a chosen value t^* ") into a closed-form expression requiring no simulation.

Verification: a ground-truth intervention was simulated directly (200,000 Monte Carlo draws), giving $E[Y^*] = 20.51$. The closed-form formula, computed using ONLY observational-data-derived parameters, gave $E[Y^*] = 20.78$. Relative error: 1.32 percent, attributable to Monte Carlo sampling noise alone.

1.5 Naive (Confounded) Estimate for Comparison

$$E[Y | T] = \exp(\text{slope} * \log(T) + 0.5 * \text{Var}(\text{residual})) \quad \text{where slope is fit directly on } \log(Y) \sim \log(T)$$

This is the WRONG way to estimate a causal effect -- it conflates the true causal path ($T \rightarrow Y$) with the confounding path ($Z \rightarrow T$ and $Z \rightarrow Y$ simultaneously). Comparing this naive slope against the causal parameter c is the mathematical definition of a 'confounding check.'

Result: naive slope = 0.0354, causal parameter c = 0.0692. The gap between these two numbers IS the confounding bias, and its size determines whether naive regression can be trusted for this specific question.

1.6 Parent-Adjustment Formula (with Proof Sketch)

$$q(x_y) = q(x_y | x_t) = \text{INTEGRAL}[p(x_y | x_t, x_{PA}(t)) * p(x_{PA}(t)) dx_{PA}(t)]$$

Proof sketch (chain-rule expansion): $q(x_y | x_t) = q(x_y, x_t) / q(x_t)$

$$= \text{INTEGRAL}[q(x) / g^*(x_t) dx_{other}] = \text{INTEGRAL}[p(x_y | x_t, x_{PA}(t)) * p(x_t | x_{PA}(t)) * p(x_{PA}(t)) * (1/p(x_t | x_{PA}(t))) dx_{PA}(t)]$$

= $\text{INTEGRAL}[p(x_y | x_t, x_{PA}(t)) * p(x_{PA}(t)) dx_{PA}(t)]$ [p(x_t | x_{PA}(t)) terms cancel]

This formula states that to compute the causal effect of T on Y, it is SUFFICIENT to average the fitted conditional p(y|t,z) over the observed (marginal) distribution of T's parents z -- rather than needing to actually intervene in the real world.

Numerical verification: an ACTUAL intervention do(T=5) was simulated on the fitted generative model (ground truth E[Y*] = 20.51). The adjustment formula, computed from observational data alone, gave 20.78 -- confirming the formula recovers the true causal quantity without ever performing an intervention.

1.7 Individual Counterfactual Formula

$$p(y^* | t, y, D) = E_{\theta} [\text{INTEGRAL } p_{\theta}(y^* | t^*, z) * p(z | t, y) dz]$$

Unlike the population-level formula in Section 1.4 (which averages Z over its PRIOR/marginal distribution), this formula answers an individual-level question: for ONE SPECIFIC observed order with known (t, y), what would its outcome have been under a different treatment t*? This requires first inferring the POSTERIOR distribution of the latent confounder z given that order's own (t, y).

1.8 Exact Joint Covariance for Counterfactuals

$$[T, Y, Y^*]^T \sim \text{Gaussian}(\mu, \Sigma) \text{ in log-space, after marginalizing out } Z$$

$$\mu = [a*\mu_Z, d*\mu_Z, b*\mu_Z - c*\log(t^*)] \text{ where } d = b - a*c \text{ } \Sigma = \begin{bmatrix} a^2*sZ2+sT2 & a*d*sZ2-c*sT2 & a*b*sZ2 \\ a*d*sZ2-c*sT2 & d^2*sZ2+c^2*sT2+sY2 & b*d*sZ2 \\ b*d*sZ2 & b*d*sZ2 & b^2*sZ2+sY2 \end{bmatrix}$$

Gaussian conditioning on this joint (standard formula: $\mu_{\text{conditional}} = \mu_{Y^*} + \Sigma_{\text{cross}} * \Sigma_{\text{observed}}^{-1} * (\text{observed} - \mu_{\text{observed}})$) gives the EXACT posterior for Y* given an order's observed (T,Y) -- no approximation needed.

Verification: this exact method and an independent two-stage method (infer posterior over Z first, then propagate it through the population formula) gave IDENTICAL numerical results to machine precision across all tested orders -- for example, both methods gave 21.1580 for one specific order's counterfactual under t*=1 hour.

1.9 Markov Equivalence / Non-Identifiability

$$\text{Graph A: } Z \rightarrow T \rightarrow Y, Z \rightarrow Y \Rightarrow \text{do}(T=5) \text{ gives } E[Y^*] = 20.78$$

$$\text{Graph B: } T \rightarrow Z \rightarrow Y, T \rightarrow Y \Rightarrow \text{do}(T=5) \text{ gives } E[Y^*] = 4.20$$

Both graphs are equally consistent with the same three-variable data. Applying the SAME intervention rule to each gives an 80 percent different answer, purely as a function of an assumption (which graph is correct) that the data alone cannot verify.

1.10 Atomic Intervention Equivalence

$$q(x_{\{-j\}} | x_j) \text{ is the same regardless of the choice of intervention distribution } g^*(x_j)$$

Verified: E[Y*|T*=5] under an atomic (fixed-value) intervention gave 20.53. Under a randomized (coin-flip between 5 and 10) intervention, filtered to the cases where T* happened to equal 5, gave 20.43 -- a 0.49 percent match, confirming atomic interventions and randomized controlled trials give the same downstream answer.

1.11 Log-Normal Algebra Identities Used Throughout

Product: $X_1 \sim \text{logN}(\mu_1, s_1^2), X_2 \sim \text{logN}(\mu_2, s_2^2) \Rightarrow X_1 X_2 \sim \text{logN}(\mu_1 + \mu_2, s_1^2 + s_2^2)$	Power: $X \sim \text{logN}(\mu, s^2) \Rightarrow X^c \sim \text{logN}(c \cdot \mu, c^2 \cdot s^2)$	Mean: $X \sim \text{logN}(\mu, s^2) \Rightarrow E[X] = \exp(\mu + 0.5 \cdot s^2)$
--	--	---

Verified numerically via Monte Carlo (200,000 samples): product identity gave empirical ($\mu=1.4994, s^2=0.2512$) versus theoretical (1.5, 0.25). Power identity gave empirical ($\mu=2.4997, s^2=0.5609$) versus theoretical (2.5, 0.5625) after correcting an initial error where the theoretical mean was mistakenly computed as μ instead of $c \cdot \mu$.

1.15 Intervention Operation on a Bayesian Network (Definition 2.1)

$q(x) = g^*(x_j \mid x_{PA^*j}) * \text{PRODUCT}_{\{i \neq j\}} p(x_i \mid x_{PA_i})$

This is the formal rule for surgically replacing one node's conditional distribution while keeping every other conditional the same. It is the mathematical definition of what it means to 'intervene' on a system, as distinct from merely observing it. The do-notation shorthand $p(x|\text{do}(X_k=c))$ is defined as an application of this rule with an atomic (Dirac-delta) replacement distribution.

1.16 Do-Calculus (Three Rules)

Rule 1 (insertion/deletion of observations): $p(y z,w,\text{do}(x)) = p(y w,\text{do}(x))$ if Y and Z are d-separated by X,W in the mutilated graph $\underline{G}_{\text{underbar}}(X)$ Rule 2 (action/observation exchange): $p(y w,\text{do}(x,z)) = p(y z,w,\text{do}(x))$ if Y and Z are d-separated by X,W in $\underline{G}_{\text{underbar}}(X) \overline{Z}$ Rule 3 (insertion/deletion of actions): $p(y w,\text{do}(x,z)) = p(y w,\text{do}(x))$ if Y and Z(W) are d-separated by X,W in $\underline{G}_{\text{underbar}}(X) \overline{Z}$
--

These three rules provide a mechanical, purely graphical algorithm for converting causal queries into purely observational quantities, without needing a full generative model. This project used a full parametric model (Sections 1.1-1.8) rather than the do-calculus directly, but the do-calculus is the standard tool for more complex graphs where specifying a complete joint distribution is impractical.

1.17 Structural Causal Model Formal Definition

An SCM is a tuple $\langle S, G, p_N \rangle$ where: G is a directed acyclic graph on variables X S is a set of structural assignments: $X_j := f_j(X_{PA(j)}, N_j)$ p_N is a distribution over independent latent noise variables N
--

A counterfactual SCM is obtained by replacing the noise distribution with its posterior given an observation: $p_N^*(n) = p_N(n \mid X=x)$. This project's twin-model approach was shown to be mathematically equivalent to this formal SCM counterfactual construction, without requiring the same measure-theoretic machinery.

1.18 Deterministic Counterfactual When Sharing All Noise (Eq. 39)

$Y^* = Y * (T / T^*)^c$ (exact, single deterministic number, when eps_Y is shared between worlds)
--

This is the OTHER extreme from the main approach used in Sections 1.7-1.8 (which shares only Z, giving a distribution over Y^*). If the residual noise term is assumed to represent a fixed, order-specific trait that persists unchanged under the counterfactual, this formula gives a single deterministic prediction instead of an expectation.

1.19 Counterfactual Non-Identifiability (Eq. 40)

Model 1: $Y = Z^b * T^c * \text{eps}_Y$ Model 2: $Y = Z^b * T^c * \text{eps}_Y^{\text{sign}(\log T)}$ Both models give the IDENTICAL observed marginal distribution of Y given Z,T (since eps_Y and eps_Y^{-1} are both $\text{logN}(0,1)$), but DIFFERENT counterfactual predictions.

This was verified directly: the two models agreed exactly when $\log(T) > 0$ ($\text{sign}=+1$, a no-op), and diverged exactly when $\log(T) < 0$ ($\text{sign}=-1$) -- confirming that without further assumptions about what the latent noise physically

represents, the counterfactual answer is fundamentally non-identifiable even when the observational model is fit perfectly.

To make Sections 1.1 through 1.11 concrete, here is one complete worked example using five real orders sampled from the dataset.

Step 1: Observed data

Order	T = approval delay (hours)	Y = delivery days (actual)
1	0.180	14.191
2	8.196	7.252
3	32.226	22.800
4	1.269	14.554
5	0.363	19.113

Step 2: Apply the fitted structural parameters

$a = -0.0695$ (Z $\rightarrow$ T effect) $b = -1.1399$ (Z $\rightarrow$ Y effect) $c = 0.0692$ (T $\rightarrow$ Y CAUSAL effect -- the number of interest)

Step 3: Compute individual counterfactuals (Section 1.7-1.8 formula) for "what if approval took only 1 hour"

Order	T_actual	Y_actual	Y* if T*=1 hour
1	0.180	14.191	21.158
2	8.196	7.252	18.693
3	32.226	22.800	30.136
4	1.269	14.554	22.817
5	0.363	19.113	24.236

Step 4: Population-level business translation (Section 1.4 formula)

$E[Y \mid \text{do}(T=24)] / E[Y \mid \text{do}(T=1)] = 24^c = 24^{0.0692} \approx 1.24$
 Interpretation: reducing average approval delay from 24 hours to 1 hour across the ENTIRE order population is predicted to reduce expected delivery time by approximately 19 percent ($1 - 1/1.24$), holding the freight-ratio distribution fixed. This is the CAUSAL claim, distinct from the naive correlational slope (0.0354), which in this dataset happens to be numerically close but is not guaranteed to remain close for other confounders or other datasets.

1.13 Assumptions Required for Each Formula to Hold

Formula	Required assumption	Consequence if violated
Eq. 1.4 (closed-form causal estimate)	Log-normal functional form is correct	Formula becomes invalid; must refit with a different parametric family
Eq. 1.6 (parent-adjustment)	No unmeasured confounders beyond Z	Estimate remains biased even though the formula is applied correctly
Eq. 1.8 (exact counterfactual)	Shared parameters theta across observed/counterfactual worlds	Counterfactual answer becomes model-dependent, not uniquely determined (Section 1.9)
Eq. 2.3 (RFF norm bound)	sin/cos trigonometric identity only -- no additional assumption needed	Holds unconditionally by construction
Eq. 2.5 (Theorem 1 boundedness)	Activations are 1-Lipschitz and homogeneous (Assumption 1 in the source theory)	Exact bounds may not hold, though qualitative boundedness often persists empirically
Eq. 3.3 (ensemble formulas)	Base models produce genuinely diverse, reasonably well-	Ensembling provides little to no benefit if one model already

	calibrated errors	dominates
--	-------------------	-----------

1.14 Summary Table of All Verified Numerical Results (Causal Section)

Quantity	Formula source	Verified value
Causal elasticity c	Section 1.1	0.0692
Naive (confounded) slope	Section 1.5	0.0354
Simulated ground-truth $E[Y^*]$	Section 1.4 verification	20.51
Closed-form formula $E[Y^*]$	Section 1.4	20.78
Relative error (formula vs simulation)	Section 1.4	1.32 percent
Graph A prediction $\text{do}(T=5)$	Section 1.9	20.78
Graph B prediction $\text{do}(T=5)$	Section 1.9	4.20
Atomic intervention $E[Y^* T^*=5]$	Section 1.10	20.53
RCT-style intervention $E[Y^* T^*=5]$	Section 1.10	20.43

Section 2 - Random Fourier Features Framework: Mathematics

2.1 The Core Transform

$\Phi(x) = \sqrt{2/D} * [\sin(\Omega^T x), \cos(\Omega^T x)]$ in $\mathbb{R}^{(2D)}$ Ω sampled ONCE from $p(\omega)$, then held FIXED throughout training

This maps any real-valued feature vector x into a new 2D-dimensional representation. D is a chosen number of random frequencies; Ω is a $(d \times D)$ matrix of frequencies drawn from a chosen distribution $p(\omega)$.

2.2 Kernel Table (choice of $p(\omega)$ determines the induced kernel)

Kernel	$k(\Delta)$	$p(\omega)$
Gaussian	$\exp(-\ \Delta\ ^2 / 2)$	Normal(0, I)
Laplacian	$\exp(-\ \Delta\ _1)$	Cauchy(0,1), per dimension
Cauchy	$\text{PRODUCT}_d 2/(1+\Delta_d^2)$	Laplace(0,1), per dimension

Result on this project's delivery-time prediction task: Gaussian kernel gave test RMSE 0.4313 (best), Laplacian gave 0.4828, Cauchy gave 0.4320.

2.3 Proof That $\|\Phi(x)\|$ Is Bounded

$\|\Phi(x)\|^2 = (2/D) * \sum_i [\sin^2(\omega_i^T x) + \cos^2(\omega_i^T x)]$
 $= (2/D) * \sum_i [1]$ since $\sin^2(a) + \cos^2(a) = 1$ for ANY a
 $= (2/D) * D = 2$ Therefore $\|\Phi(x)\| = \sqrt{2}$ EXACTLY, for every x , regardless of scale.

This is not an approximate or empirical bound -- it is a deterministic algebraic identity. Verified numerically across input scales from 1 to 10,000 (four orders of magnitude): the transformed norm remained fixed at 1.4142 in every single case.

2.4 Neural Tangent Kernel and the Telescoping SGD Approximation

$f_{\theta^T}(x) = f_{\theta^0}(x) - \sum_t \sum_i K_t(x, x_i) * g_{it}$ where $K_t(x, x_i) = \text{grad}_{\theta} f_{\theta_{t-1}}(x)^T * \text{grad}_{\theta} f_{\theta_{t-1}}(x_i)$

This reframes a trained neural network's output as a KERNEL MACHINE, where K_t is the Neural Tangent Kernel at training step t . This equation is the conceptual bridge that lets kernel theory (boundedness, conditioning) describe deep-learning training dynamics.

2.5 Theorem 1 - Kernel Boundedness (Two Parts)

Part 1 (raw features): $\sup_{\{x,x'\}} K_t(x,x') = \text{infinity}$ [UNBOUNDED] Part 2 (RFF features): $K_{\text{tilde}_t}(x,x') = \gamma_t(x,x') + m_t(x,x')$ both γ_t and m_t are BOUNDED

Verified via a hand-coded 2-layer ReLU network with manually-derived backpropagation gradients (no autodiff library used). Numerical gradient checking (finite-difference method) confirmed the manual gradients were exact, e.g. $\text{dW1}[0,6]$: finite-difference estimate -0.002172 versus analytic -0.002172, a 0.000000 percent relative error.

Kernel value $K(x,x)$ measured at five input scales:

$\ x\ $	$K_{\text{raw}}(x,x)$ [unbounded]	$K_{\text{RFF}}(x,x)$ [bounded]
1	8.05	1.99
10	663.2	2.01
100	66,180	2.12
1,000	6,618,000	2.18
10,000	661,800,000	2.24

2.6 Exact Decomposition Proof (Eq. 25)

$\text{grad_theta } f(\text{Phi}(x)) = [\text{grad_psi } g_{\text{psi}}(h1), \text{vec}(\text{grad_h1 } f(\text{Phi}(x)) * \text{Phi}(x)^T)]$
 $K_{\text{tilde}_t}(x,x') = \text{grad_psi}(h1)^T * \text{grad_psi}(h1') \leftarrow \gamma_t \text{ term}$
 (bounded, upper layers) $+ [\text{grad_h1 } f(x)]^T * [\text{grad_h1 } f(x')] * \text{Phi}(x)^T * \text{Phi}(x') \leftarrow m_t \text{ term (RFF-kernel-like)}$

Verified as an EXACT identity (not an approximation) by computing γ_t and m_t explicitly from real gradients and confirming $\gamma_t + m_t$ equals the true full kernel to six decimal places, for every tested input pair. An initial implementation attempt failed this exact-match check by a small margin, tracing to a missing ReLU-derivative mask term in the m_t computation -- corrected before reporting the final verified result.

2.7 Boundedness of the γ_t Term (Eq. 26-27)

$\|h1\| \leq \sqrt{2} * \|W0\|_2$ (since $\|\text{Phi}(x)\| \leq \sqrt{2}$ and $h1 = \text{ReLU}(W0 * \text{Phi}(x)) \Rightarrow |\gamma_t(x,x')| \leq B < \text{infinity}$ for some constant B depending only on network weights

Verified: $\|h1\|$ stayed in the range 0.42 to 0.59 regardless of input scale spanning 1 to 10,000 (theoretical cap: $\sqrt{2} * \|W0\|_2 = 2.1057$).

2.8 Rahimi and Recht Kernel Approximation Theorem

$E_{\omega}[\text{zeta}_{\omega}(x) * \text{zeta}_{\omega}(y)] = k(x - y)$ where $\text{zeta}_{\omega}(x) = \exp(i * \omega^T * x)$ [Bochner's theorem]

This is the foundational result RFF is built on. Verified via Monte Carlo: as the number of sampled frequencies D increased from 4 to 4096, the empirical inner product $\text{Phi}(x)^T * \text{Phi}(x')$ converged toward the true kernel value (scaled by a constant factor of 2, arising from this project's specific $\sqrt{2/D}$ normalization choice), with absolute error shrinking from 0.067 (D=4) to 0.007 (D=4096).

2.9 Homogeneity Lemmas (Appendix A)

Lemma 1: $\text{grad_hk } f(\alpha * x) = \alpha^{(s^{(m-k)} - 1)} * \text{grad_hk } f(x)$ Theorem 2: $\|\text{grad_theta } f(x)\| \leq C1 * \|x\|$ [upper bound]
 $\|\text{grad_theta } f(\alpha * x)\| \geq C2 * \alpha^{(s^m - 1)}$ [lower bound, $\alpha > 1$]

Verified using a zero-bias ReLU network (exactly homogeneous, since ReLU is 1-homogeneous, $s=1$). Confirmed: $f(\alpha * x) = \alpha * f(x)$ exactly (to 6 decimal places, $\alpha=1$ to 100). $\|\text{grad_h1 } f(\alpha * x)\|$ stayed EXACTLY constant (0.559222 unchanged) confirming Lemma 1's exponent collapses to zero when $s=1$. $\|\text{grad_theta } f(x)\| / \|x\|$ stayed EXACTLY constant (2.572466), confirming Theorem 2's upper bound is achieved with EQUALITY in this special case.

2.14 MLP Forward Pass Definition (Eq. 2-3)

$$h^{(k+1)} = \text{sigma}_k(W_k h^k), \quad h^0 = x, \quad y_{\text{hat}} = f_{\text{theta}}(x) = h^m$$

Standard feed-forward network notation: each layer applies a weight matrix followed by a non-linearity. $\text{theta} = \{W_k\}$ collects all weight matrices. This notation is the basis for every subsequent gradient and kernel formula in this section.

2.15 Lemma 2: Forward and Backward Norm Bounds

$$\text{Define } S_k = \text{PRODUCT}_{i=0}^{k-1} \|W_i\|_2, \quad T_k = \text{PRODUCT}_{i=k}^{m-1} \|W_i\|_2 \quad \text{Then:} \quad \|h_k\| \leq S_k * \|x\| \quad (\text{forward activation norm bound})$$

$$\| \text{grad}_{h_k} f_{\text{theta}}(x) \| \leq T_k \quad (\text{backward gradient norm bound})$$

These bounds are expressed directly in terms of the network's own weight matrix norms, giving explicit, computable constants rather than a vague 'it's bounded' statement -- useful for diagnosing which specific layer's weights are contributing most to potential instability.

2.16 Theorem 2: Two-Sided Gradient Growth Bound

$$\text{Upper bound: } \| \text{grad}_{\text{theta}} f_{\text{theta}}(x) \| \leq C_1 * \|x\| \quad \text{for a constant } C_1 \text{ depending on network weights}$$

$$\text{Lower bound: } \| \text{grad}_{\text{theta}} f_{\text{theta}}(\alpha * x) \| \geq C_2 * \alpha^{(s^m - 1)} \quad \text{for } \alpha > 1, C_2 \text{ independent of } \alpha$$

This pins down the growth rate of the raw-feature gradient between a best case (linear growth, upper bound) and a guaranteed-at-least-this-fast growth rate (lower bound) tied to the network's homogeneity degree. Verified for our zero-bias ReLU test network: the ratio $\| \text{grad}_{\text{theta}} f(x) \| / \|x\|$ stayed EXACTLY constant at 2.572466 across two orders of magnitude of input scale -- confirming the upper bound is achieved with equality in this exactly-homogeneous special case.

Taking one standardized feature vector x_0 from the training set and scaling it by increasing factors α , both the raw scaled vector's norm and its RFF-transformed norm were measured directly:

α	$\ \alpha * x_0\ $ (raw)	$\ \Phi(\alpha * x_0)\ $ (RFF)	sqrt(2) reference
1	1.00	1.4142	1.4142
10	10.00	1.4142	1.4142
100	100.00	1.4142	1.4142
1,000	1,000.00	1.4142	1.4142
10,000	10,000.00	1.4142	1.4142

The raw norm grows exactly linearly with α , as expected. The RFF-transformed norm remains fixed at exactly $\sqrt{2}$ in every single case -- confirming Section 2.3's algebraic identity holds exactly in practice, not merely approximately.

2.11 Manual Gradient Derivation, Step by Step

For a 2-layer ReLU network with layers $h_1 = \text{ReLU}(W_0 * x)$, $h_2 = \text{ReLU}(W_1 * h_1)$, $y = W_2 * h_2$, the gradients were derived by hand as follows (no automatic differentiation library used):

$$\begin{aligned} dL/dW_2 &= h_2^T && (\text{gradient wrt output layer weights}) \\ dL/dh_2 &= W_2 && (\text{gradient flowing backward into } h_2) \\ dL/dz_2 &= dL/dh_2 * (z_2 > 0) && (\text{apply ReLU derivative mask at layer 2}) \\ dL/dW_1 &= \text{outer}(h_1, dL/dz_2) && (\text{gradient wrt hidden layer 1 weights}) \\ dL/dh_1 &= dL/dz_2 @ W_1^T && (\text{gradient flowing backward into } h_1) \\ dL/dz_1 &= dL/dh_1 * (z_1 > 0) && (\text{apply ReLU derivative mask at layer 1}) \\ dL/dW_0 &= \text{outer}(x, dL/dz_1) && (\text{gradient wrt input layer weights}) \end{aligned}$$

This chain was implemented directly in NumPy with no autodiff. The correctness of this manual derivation was independently confirmed via a finite-difference numerical gradient check: perturbing a single weight ($W_1[0,6]$) by a

small epsilon and measuring the resulting change in output gave -0.002172, exactly matching the manually-derived analytic gradient of -0.002172 (0.000000 percent relative error).

2.12 Assumptions Required for the RFF Theorems

Assumption	Where used	Consequence if violated
Activation is 1-Lipschitz	Lemma 2 (norm bounds)	The forward/backward norm bounds in Section 2.7 no longer hold as stated
Activation is s-homogeneous	Lemma 1, Theorem 2	Exact exponents in the growth-rate formulas change; qualitative unboundedness still typically holds
Network is non-degenerate (f is not constant)	Theorem 2 lower bound	The lower-bound growth guarantee becomes vacuous
Omega sampled from the TRUE Fourier dual of the target kernel	Lemma 3 (Rahimi-Recht)	$\Phi(x)^T \Phi(x')$ no longer approximates the intended kernel $k(x-x')$
D (number of frequencies) sufficiently large	Lemma 3 convergence	High Monte Carlo variance in the kernel approximation; verified empirically that error shrinks from 0.067 (D=4) to 0.007 (D=4096)

2.13 Summary Table of All Verified Numerical Results (RFF Section)

Quantity	Formula source	Verified value
Gaussian-kernel test RMSE	Section 2.2	0.4313 (best of 3 kernels tested)
RFF feature norm (all scales)	Section 2.3, 2.10	1.4142 (exactly $\sqrt{2}$, constant)
Raw kernel value at $\ x\ =1$	Section 2.5	8.05
Raw kernel value at $\ x\ =10,000$	Section 2.5	661,800,000
RFF kernel value at $\ x\ =1$	Section 2.5	1.99
RFF kernel value at $\ x\ =10,000$	Section 2.5	2.24
Gradient check relative error	Section 2.11	0.000000 percent
$\gamma_t + m_t$ vs exact K_{tilde} match	Section 2.6	Match to 6 decimal places

Section 3 - Forecast-Then-Optimize Framework: Mathematics

3.1 Evaluation Metric

$$\text{SMAPE} = 100 * \text{mean}(2 * |\text{forecast} - \text{actual}| / (|\text{actual}| + |\text{forecast}|))$$

This metric is bounded in $[0, 200]$ by construction, and normalizes error by the scale of the values being compared, making error magnitudes comparable across periods with very different absolute revenue levels.

3.2 Preprocessing Formulas

Min-Max Scaling: $x_{\text{scaled}} = (x - \min(x)) / (\max(x) - \min(x))$ -> bounded to $[0,1]$
 Standardization: $x_{\text{scaled}} = (x - \text{mean}(x)) / \text{std}(x)$
 -> mean 0, std 1
 Log-transform: $y_{\text{transformed}} = \log(1 + y)$
 -> compresses right-skew
 One-hot encoding: day_of_week (0-6) -> 7 binary columns, exactly one =1 per row

3.3 Ensemble Aggregation Formulas

Mean: $y_{\text{hat}} = (1/K) * \sum_i M_i(x)$
 Median: $y_{\text{hat}} = \text{median}(M_1(x), \dots, M_K(x))$
 Quantile-q: $y_{\text{hat}} = \text{quantile}_q(M_1(x), \dots, M_K(x))$, q in $[0,1]$
 Weighted median: cumulative-weight midpoint, weights = $\text{softmax}(-\text{recent_validation_error})$

Verified result: quantile-0.9 ensemble achieved SMAPE 11.68 percent, the single best result across the entire forecasting project, beating every individual base model and every other combination method tested.

3.3b Bayesian Model Averaging (BMA) Weight Formula

$\text{weight}_i \propto P(\text{Data} \mid \text{Model}_i) \quad (\text{posterior model probability})$ Practical approximation used in this project: $\text{weight}_i \propto 1 / (\text{validation_error}_i + \epsilon)$

BMA is a principled, probabilistic generalization of inverse-error weighting: rather than an ad-hoc heuristic, it gives calibrated, uncertainty-aware model combination weights. This project implemented the practical inverse-validation-error approximation rather than computing full posterior model probabilities, since the latter requires a specified prior and likelihood for each candidate model.

$\text{weights}_t = \text{softmax}(-\text{recent_error}[k]) \quad \text{computed on a SLIDING WINDOW of the } W \text{ most recent errors}$ $\hat{y}_t = \sum_k \text{weights}_t[k] * M_k(x_t)$
--

Unlike a static weighted ensemble (weights fixed once from a validation window), this recomputes weights as new data arrives, adapting to regime shifts. Result: SMAPE 15.02 percent, versus 17.13 percent for the static-weighted variant.

3.5 Direct Loss Estimator (Residual Learning) Formula

$\hat{y}_{\text{corrected}} = \hat{y}_{\text{base}} + g(\text{features}) \quad \text{where } g \text{ is trained to predict } (\text{actual} - \hat{y}_{\text{base}})$
--

Result: SMAPE 44.01 percent, WORSE than the uncorrected base model's 12.55 percent. With only 28 validation points, the secondary residual model overfit noise rather than genuine error structure -- an honest negative finding retained in this report rather than smoothed over.

3.6 FFORMA-Style Instance-Specific Weighting Formula

$\text{weights}(x) = \text{normalize}([\text{model}_1(\text{features}(x)), \dots, \text{model}_K(\text{features}(x))])$ each model_k predicts an instance-specific weight $\hat{y} = \sum_k \text{weights}(x)[k] * M_k(x)$
--

Result: SMAPE 17.46 percent, also worse than the best single base model, for the same limited-data reason as the Direct Loss Estimator above.

3.7 Conformal Prediction Interval Formula

$\text{radius} = \text{quantile}(\text{validation_residuals} , 1 - \alpha) \quad \text{interval} = [\text{forecast} - \text{radius}, \text{forecast} + \text{radius}]$
--

This produces a distribution-free interval with a provable finite-sample coverage guarantee. Result: target coverage 90 percent ($\alpha=0.10$); radius +/- 18,928; empirical coverage achieved 100 percent -- technically valid (never under-covers) but wider than necessary given the small calibration set used.

3.8 DLinear / NLinear Formulas

NLinear: $\hat{y} = \text{LinearLayer}(x - x_{\text{last}}) + x_{\text{last}}$ DLinear: $\hat{y} = \text{LinearLayer}(\text{trend}(x)) + \text{LinearLayer}(\text{seasonal}(x))$, $\text{seasonal}(x) = x - \text{trend}(x)$ trend(x) computed via a moving average (Simple Moving Average, kernel size 7 days)

Result: NLinear SMAPE 24.18 percent, DLinear SMAPE 36.70 percent -- both underperformed the explicit-calendar-feature Linear Trend model (12.55 percent) in this project, since neither has an explicit day-of-week input and must infer weekly seasonality implicitly from the window shape.

3.9 RevIN (Reversible Instance Normalization) Formula

Normalize:	$x_{\text{norm}} = (x - \text{mean}(\text{window})) / \text{std}(\text{window})$	computed PER WINDOW,
not globally Model:	$y_{\text{norm_hat}} = \text{LinearLayer}(x_{\text{norm}})$	Denormalize: $y_{\text{hat}} = y_{\text{norm_hat}} * \text{std}(\text{window}) + \text{mean}(\text{window})$

Result: SMAPE 19.29 percent -- better than NLinear (24.18) and much better than no normalization at all (27.34), confirming per-instance normalization measurably helps window-based linear models on this data.

3.10 Full Constraint List for This Section

Constraint	Rule
Non-negativity of price	price > 0
Non-negativity of delivery time	delivery_days > 0
Upper bound on delivery time	delivery_days < 100
Bounded approval delay	0 <= approval_delay_hours < 240
Forecast non-negativity	prediction = max(prediction, 0)
Ensemble weight validity	sum(weights) = 1, weights >= 0
Quantile bound	0 <= q <= 1
RFF frequency matrix fixed	Omega set once, never updated during training
Conformal target coverage	1 - alpha (e.g. 90 percent)

3.11 Complete Verified Results Leaderboard (All Methods, This Section)

Rank	Method	SMAPE (percent)
1	Quantile-0.9 Ensemble (Section 3.3)	11.68
2	Linear Trend (single model)	12.55
3	Quantile-0.75 Ensemble	12.45
4	MLP on lag features (single model)	18.10
5	RevIN + Linear (Section 3.9)	19.29
6	NLinear (Section 3.8)	24.18
7	Naive Seasonal (single model)	26.65
8	Plain window-linear, no normalization	27.34
9	Mean Ensemble (Section 3.3)	16.37
10	Median / Weighted-Median Ensemble	15.52
11	Dynamic Weighting Ensemble (Section 3.4)	15.02
12	Static BMA-like weighted ensemble	17.13
13	FFORMA-style instance weighting (Section 3.6)	17.46
14	TCN-style (dilated causal convolution)	34.45
15	DLinear (Section 3.8)	36.70
16	Direct Loss Estimator (Section 3.5)	44.01

3.12 Assumptions Required for the Forecasting Formulas

Formula	Required assumption	Consequence if violated
Ensemble formulas (Section 3.3)	Base models have diverse, roughly comparable error levels	Ensembling provides minimal benefit if one model already dominates -- observed directly in this project (Mean Ensemble at

		16.37 percent underperformed the single best base model at 12.55 percent)
DLE / FFORMA-style (Sections 3.5-3.6)	Sufficient validation data to fit a reliable secondary/meta-model	Meta-learner overfits noise -- observed directly (DLE reached 44.01 percent, worse than the uncorrected 12.55 percent baseline, using only 28 validation points)
Conformal interval (Section 3.7)	Calibration set is representative of the test distribution	Interval remains technically valid but can become uninformatively wide -- observed directly (100 percent empirical coverage against a 90 percent target)
DLinear / NLinear (Section 3.8)	Target series' seasonality can be adequately inferred from a raw window shape alone	Underperforms models with explicit calendar features when seasonality is strong and regular -- observed directly in this project

3.13 Cross-Section Verification Method Summary

Across Sections 1, 2, and 3, every formula presented was checked using at least one of the following methods, rather than being accepted on the basis of the source theory alone:

- Exact algebraic identity, checked to machine precision (used for the RFF norm bound, the twin-model/two-stage counterfactual equivalence, and the kernel decomposition identity)
- Closed-form result cross-checked against independent Monte Carlo simulation (used for the causal intervention formula and the parent-adjustment formula)
- Finite-difference numerical gradient checking against manually-derived analytic gradients (used for the hand-coded neural network backpropagation)
- Direct empirical replication on real data, with results reported honestly even when they diverged from expectation (used throughout the forecasting section, e.g. the DLE and FFORMA-style meta-learner underperformance)

3.14 Two Implementation Errors Caught During This Verification Process

In the interest of a complete and honest mathematical record, two specific implementation errors were identified and corrected during the verification work described above, rather than only after the fact:

- Error 1: an early check of the log-normal power-law identity (Section 1.11) printed an incorrect theoretical mean, using μ instead of the correct $c \cdot \mu$. This was caught because the resulting discrepancy was large and systematic rather than resembling random noise.
- Error 2: an early implementation of the kernel decomposition identity (Section 2.6) did not exactly match the true full gradient, differing by a small but consistent amount. This was traced to a missing ReLU-derivative mask term in the computation of the m_t component, and was corrected before the final exact-match result was reported.